

QUALIFAST BUILDINGS

Sistema de Gestión de Comisaría Inteligente (IoT)

Memoria Técnica Final de Proyecto

Álvaro López · Daniel Nicolás Ramírez · Daniel Vicente · Fernando Fernández
Ingeniería de Software · Ámbito IoT
Python / Flet / MySQL / ESP32 / Arduino C++

ÍNDICE

ÍNDICE.....	2
1. RESUMEN EJECUTIVO.....	4
2. INTRODUCCIÓN, CONTEXTO Y ESTADO DEL ARTE.....	4
2.1 Contexto y Justificación.....	4
2.2 Estado del Arte.....	4
2.3 Planteamiento del Problema.....	4
3. OBJETIVOS DEL PROYECTO.....	5
3.1 Objetivo General.....	5
3.2 Objetivos Técnicos Específicos.....	5
3.3 Objetivo Social e Inclusivo.....	5
4. METODOLOGÍA DE TRABAJO.....	6
4.1 Marco de Gestión: Scrum Adaptado.....	6
4.2 Herramientas.....	6
4.3 Distribución de Roles.....	6
5. DESARROLLO TÉCNICO.....	6
5.1 Infraestructura de Hardware y Firmware IoT.....	6
5.1.A Montaje del Circuito y Mapeo de Pines (ESP32).....	6
5.1.B Integración del Sistema de Videovigilancia (ESP32-CAM).....	7
5.2 Capa de Persistencia: Migración a MySQL.....	7
5.3 Refactorización Arquitectónica: DAO + Patrón Fachada.....	7
5.3.A Descomposición del Monolito.....	7
5.3.B Patrón Fachada (Facade Pattern).....	8
5.4 Configuración, Tolerancia a Fallos y Diagnóstico.....	8
5.4.A Desacoplamiento de la Configuración (ajustes_locales.json).....	8
5.4.B Interfaz de Diagnóstico en el Login.....	8
5.4.C Configuración de Hardware en Caliente (Hot-Swap IPs).....	8
5.5 Videovigilancia Resiliente y Monitorización de Hardware.....	9
5.5.A Sistema de Tolerancia a Fallos en el Módulo CCTV.....	9
5.5.B Sistema de Monitorización de Salud del Hardware (Heartbeat).....	9
5.6 Funcionalidades Avanzadas.....	9
5.6.A Control Dual de Actuadores (Automático / Manual).....	9
5.6.B Auditoría y Exportación de Datos (CSV).....	9
5.6.C Eficiencia Energética: Cálculo Determinista del Consumo.....	9
5.6.D Heartbeat Elástico (Ajuste Dinámico de Responsividad).....	10
5.7 Concurrencia, UX y Optimizaciones.....	10
5.7.A Controlador UI Optimizado mediante Threading.....	10
5.7.B Mejoras de Diseño Visual y Ergonomía.....	10
5.8 Calidad de Código y Repositorio.....	10
5.8.A Manejo de Excepciones y Logging (Production-Ready).....	10
5.8.B Optimización del Repositorio Git.....	11

5.8.C Aislamiento de Microservicios (servicios_iot/)	11
5.9 Optimización de Base de Datos: Keyset Pagination	11
5.10 Reestructuración del Subsistema de Chat	11
5.10.A Refactorización Visual — Mensajería Moderna	11
5.10.B Gestión del Ciclo de Vida del Dato: Borrado de Historial	12
5.11 Ciberseguridad: Blindaje BCrypt	12
5.11.A Reparación de Vulnerabilidad en el Proceso de Login	12
5.11.B Prevención de Corrupción de Datos (Doble Cifrado)	12
5.12 Limpieza de Arquitectura y Sincronización	12
5.12.A Eliminación del Código Zombie	12
5.12.B Sincronización del Gemelo Digital con el Hardware Real	13
6. VALOR DIFERENCIAL E INNOVACIÓN	13
6.1 Diseño Hexagonal Sincronizado con la Maqueta Real	13
6.2 Keyset Pagination: Ingeniería de Rendimiento Aplicada	13
6.3 Interfaz Glassmorphism: Usabilidad en Entornos de Alta Tensión	13
6.4 Tolerancia a Fallos: Circuit Breaker Implícito	13
7. ESTUDIO ECONÓMICO Y MATERIALES	14
7.1 Coste de Hardware Electrónico (BOM — Bill of Materials)	14
7.2 Coste de Construcción de la Maqueta Física	14
7.3 Coste de Desarrollo de Software (Horas de Ingeniería)	14
7.4 Resumen del Presupuesto Total	15
8. CONCLUSIONES, AUTOCRÍTICA Y LÍNEAS FUTURAS	15
8.1 Conclusiones	15
8.2 Autocrítica y Limitaciones	15
8.3 Líneas de Trabajo Futuro	16
9. BIBLIOGRAFÍA Y REFERENCIAS	16

1. RESUMEN EJECUTIVO

Qualifast Buildings es un sistema integral de gestión inteligente para comisarías, desarrollado como proyecto final de grado por cuatro ingenieros de software. Transforma una instalación policial convencional en una infraestructura IoT completamente monitorizada y automatizada, combinando hardware embebido real (ESP32), firmware propio en Arduino C++ y una aplicación de escritorio corporativa sobre arquitectura MVC+DAO+Fachada en Python con interfaz gráfica Flet.

El sistema integra cinco subsistemas principales:

- Red de sensores IoT en tiempo real: temperatura, humedad, calidad del aire, gas y luminosidad.
- Videovigilancia con streaming MJPEG vía ESP32-CAM con tolerancia a fallos.
- Automatización de actuadores (ventilación e iluminación) con modo dual automático/manual.
- Gestión de reclusos, celdas y personal sobre base de datos MySQL normalizada.
- Subsistema de comunicaciones internas cifradas con confirmación de lectura tipo WhatsApp.

Los logros técnicos más destacados son la paginación Keyset de coste $O(1)$, el sistema de autenticación BCrypt blindado contra vulnerabilidades de prefijo y doble cifrado, y la interfaz Glassmorphism con plano hexagonal sincronizado con la maqueta física real.

2. INTRODUCCIÓN, CONTEXTO Y ESTADO DEL ARTE

2.1 Contexto y Justificación

Las comisarías operan en su mayoría con sistemas analógicos o herramientas de gestión fragmentadas. El Plan de Digitalización de las Administraciones Públicas 2021-2025 del Gobierno de España reconoce esta brecha, pero el sector de seguridad pública permanece rezagado respecto a la industria privada en la adopción de IoT y BMS (Building Management Systems).

Qualifast Buildings responde a esta brecha con una solución de bajo coste (menos de 75€ de hardware), código abierto y alta modularidad, diseñada específicamente para el entorno policial.

2.2 Estado del Arte

Plataformas de referencia como Home Assistant, OpenHAB o AWS IoT Core demuestran la madurez del paradigma de integración de sensores y actuadores sobre protocolos estándar (MQTT, HTTP/REST). Este proyecto implementa su propia API HTTP ultraligera en el ESP32, sin dependencia de servicios en la nube, lo que garantiza operación autónoma en redes locales con requisitos estrictos de privacidad.

En el plano arquitectónico, se aplican los patrones MVC (Fowler, 2002) y DAO (Oracle J2EE Guide) como estándar de facto para aplicaciones empresariales complejas. Para la seguridad de credenciales se emplea BCrypt (Provos y Mazières, 1999), función de hash adaptativa diseñada para resistir ataques de fuerza bruta. Para la eficiencia de consultas históricas, se implementa Keyset Pagination (Winand, 2012), con coste de recuperación $O(1)$ independiente del volumen acumulado.

2.3 Planteamiento del Problema

La ausencia de sistemas de monitorización unificada en entornos policiales obliga al personal a realizar rondas físicas periódicas para verificar el estado de celdas, accesos, calidad del aire e iluminación. Esto genera ineficiencias operativas, riesgos para la salud y nula trazabilidad histórica de eventos críticos.

Pregunta de investigación: ¿Es posible diseñar un sistema IoT de bajo coste, alta seguridad y arquitectura modular que cubra de forma integrada la monitorización ambiental, el control de accesos, la videovigilancia y la gestión administrativa de una instalación policial real? La respuesta es afirmativa y este proyecto lo demuestra.

3. OBJETIVOS DEL PROYECTO

3.1 Objetivo General

Desarrollar un sistema integral de gestión inteligente para comisarías que integre, en una única plataforma, la monitorización IoT en tiempo real, el control de actuadores, la videovigilancia, la administración de personal y reclusos, y las comunicaciones internas cifradas; garantizando modularidad, seguridad y trazabilidad total de eventos.

3.2 Objetivos Técnicos Específicos

- Diseñar e implementar el firmware IoT del ESP32 con API HTTP embebida para cinco sensores (DHT22, LDR, MQ-2, MQ-135) y control de actuadores de bajo y alto voltaje.
- Migrar la capa de persistencia de ficheros JSON planos a MySQL normalizado con soporte ACID y pool de conexiones.
- Refactorizar la arquitectura software siguiendo patrones MVC, DAO y Fachada para garantizar separación de responsabilidades y mantenibilidad.
- Implementar autenticación BCrypt blindada contra vulnerabilidades de prefijo y corrupción por doble cifrado.
- Desarrollar algoritmo de paginación Keyset (cursor-based) para históricos de sensores y actuadores con coste $O(1)$.
- Integrar videovigilancia MJPEG con tolerancia a fallos de red y reconexión automática.
- Construir interfaz Glassmorphism con plano hexagonal sincronizado con la maqueta física.

3.3 Objetivo Social e Inclusivo

El sistema incorpora una perspectiva de impacto social desde el diseño:

Salud y derechos fundamentales: El sensor MQ-135 monitoriza CO_2 y compuestos orgánicos volátiles en celdas de reclusos. La automatización del sistema de ventilación ante niveles críticos garantiza el derecho a un entorno saludable de acuerdo con el artículo 3 del CEDH, eliminando la dependencia del factor humano en la protección de este derecho.

Accesibilidad universal: Indicadores de estado con alto contraste (verde/rojo con texto), iconos con etiquetas descriptivas, y navegación por teclado con autofocus y orden de tabulación lógico. Diseño accesible para personas con daltonismo (8% de la población masculina) y con movilidad reducida en manos.

Inclusión profesional: El diagnóstico visual del login y la configuración en caliente de IPs permiten que cualquier agente opere el sistema sin conocimientos técnicos, eliminando la dependencia de personal TI especializado para el mantenimiento rutinario.

4. METODOLOGÍA DE TRABAJO

4.1 Marco de Gestión: Scrum Adaptado

El proyecto se estructuró en dos fases o releases principales, cada una subdividida en sprints quincenales. Al inicio de cada sprint, reunión de Sprint Planning para distribuir tareas del Product Backlog; al final, Sprint Review para validar entregables funcionales. Equipo de cuatro integrantes con dedicación académica.

4.2 Herramientas

Herramienta	Uso
Trello	Product Backlog y seguimiento mediante tablero Kanban (Por hacer / En progreso / En revisión / Completado)
GitHub	Control de versiones con ramas por funcionalidad (feature branches), pull requests y .gitignore configurado
Visual Studio Code	Entorno de desarrollo principal. Extensiones: Pylance, Python Debugger, GitLens y Remote SSH
WhatsApp / Meet	Comunicación diaria del equipo y coordinación de sesiones presenciales y remotas

4.3 Distribución de Roles

Integrante	Rol Principal	Subsistema
Álvaro López	Tech Lead / Arquitectura	MVC, DAO, Fachada, BD
Daniel Nicolás Ramírez	Backend & Seguridad	BCrypt, Keyset, DAOs
Daniel Vicente	Hardware / Firmware IoT	ESP32, Arduino C++, sensores
Fernando Fernández	Frontend / UX-UI	Flet, Glassmorphism, Chat

5. DESARROLLO TÉCNICO

— FASE 1 — Arquitectura, Hardware y Mejoras de Sistema

5.1 Infraestructura de Hardware y Firmware IoT

5.1.A Montaje del Circuito y Mapeo de Pines (ESP32)

 PROBLEMA	En la Fase 1, el sistema carecía de conexión física; los datos de la comisaría eran simulados por software.
 IMPLEMENTACIÓN	Se diseñó un circuito físico con ESP32 (Arduino C++). Entradas: DHT22 (Pin 4), LDR (Pin 32), MQ-2 (Pin 33). Salidas: ventilación (Pin 12), iluminación (Pin 13), control de accesos (Pines 14/15). Se levantó un servidor web embebido (WebServer) con API HTTP: /sensores (JSON) y /actuadores (órdenes externas).
 RESULTADO	El sistema pasó de conceptual a un entorno embebido real con una API HTTP ultraligera.

Tabla de Mapeo de Pines — ESP32:

Componente	Tipo	GPIO	Alimentación	Notas
DHT22	Entrada Digital	GPIO 4	3.3 V	Temperatura / Humedad
LDR (Luz)	Entrada Analógica	GPIO 32	3.3 V (Div. tensión)	Luminosidad
MQ-2 (Humo)	Entrada Analógica	GPIO 33	5 V (VIN)	Humo / Gas
LEDs (Iluminación)	Salida Digital	GPIO 13	Desde el pin	Iluminación celdas
Ventilador 5V	Salida Digital	GPIO 12	5 V (Vía transistor)	Motor DC / ventilación
Motor 12V	Salida Digital	GPIO 14	12 V (Vía relé)	Control de accesos / celdas

⚠ Los pines con 5 V o 12 V no se alimentan directamente desde el GPIO. Los de 5 V usan transistor NPN (BC547/2N2222) y los de 12 V un módulo relé; el GPIO actúa solo como señal de disparo.

5.1.B Integración del Sistema de Videovigilancia (ESP32-CAM)

 PROBLEMA	La monitorización perimetral requería transmisión de vídeo en directo; el ESP32 estándar carece de hardware para ello.
 IMPLEMENTACIÓN	Se incorporó un módulo ESP32-CAM con sensor OV2640 configurado para streaming MJPEG (Motion JPEG) en el puerto 81.
 RESULTADO	Separación de responsabilidades en hardware: ESP32 gestiona telemetría y actuadores; ESP32-CAM se dedica en exclusiva al vídeo, evitando cuellos de botella.

5.2 Capa de Persistencia: Migración a MySQL

 PROBLEMA	Los datos se almacenaban en archivos JSON planos, generando problemas críticos de concurrencia, nula escalabilidad histórica y riesgo de corrupción.
 IMPLEMENTACIÓN	Migración a MySQL con estructura normalizada (usuarios, personas, presos, celdas, sensores, sensores_log, actuadores, historico_actuadores). Se implementó un Pool de Conexiones (MySQLConnectionPool) con connect_timeout=2 para evitar bloqueos de hilo.
 RESULTADO	El sistema soporta miles de inserciones históricas, permite auditorías cruzadas y garantiza persistencia bajo el estándar ACID.

5.3 Refactorización Arquitectónica: DAO + Patrón Fachada

5.3.A Descomposición del Monolito

 PROBLEMA	El archivo manejador_datos.py era un monolito de más de 600 líneas que mezclaba autenticación, telemetría IoT, actuadores y chat. Cualquier cambio implicaba riesgo de ruptura en cadena.
 IMPLEMENTACIÓN	Aplicación del Principio de Responsabilidad Única mediante el Patrón DAO. Se creó el paquete modelo/dao/ con 4 módulos: conexion_db.py (ciclo de vida de la conexión), dao_usuarios.py (CRUD y autenticación), dao_iot.py (telemetría y actuadores) y dao_chat.py (mensajería interna).
 RESULTADO	Modularidad extrema. Si cambia la BD a PostgreSQL, solo se modifica el DAO afectado; el resto permanece intacto.

5.3.B Patrón Fachada (Facade Pattern)

 PROBLEMA	Dividir el monolito en 4 módulos habría provocado un fallo masivo en cadena en main.py y las vistas, que buscaban funciones en el antiguo módulo.
 IMPLEMENTACIÓN	manejador_datos.py se vació y transformó en un enrutador limpio que expone funciones mediante importación selectiva de los DAOs. Flujo: [Vistas / main.py] → [manejador_datos.py (Fachada)] → [dao_usuarios, dao_iot, dao_chat].
 RESULTADO	Modularidad sin romper una sola línea de código externo.

5.4 Configuración, Tolerancia a Fallos y Diagnóstico

5.4.A Desacoplamiento de la Configuración (ajustes_locales.json)

 PROBLEMA	Credenciales de BD e IPs del hardware estaban hardcodeadas. Si la BD estaba caída, el programa se congelaba al iniciar.
 IMPLEMENTACIÓN	Sistema de persistencia local mediante ajustes_locales.json: si no existe, se crea con valores por defecto. Se añadió connect_timeout=2 al Pool de MySQL para forzar aborto rápido sin bloquear el hilo principal.

 RESULTADO	El programa arranca instantáneamente sin conexión de red. El código respeta la separación entre código y configuración, eliminando conflictos de Git.
--	---

5.4.B Interfaz de Diagnóstico en el Login

 PROBLEMA	El usuario no sabía si había conexión a la BD hasta que el login fallaba. Modificar la IP requería editar el código fuente.
 IMPLEMENTACIÓN	Se integró en vista_login.py un indicador visual (Verde=Conectado / Rojo=Desconectado) actualizado cada 3 segundos mediante un hilo de "ping". Un botón de engranaje despliega un AlertDialog para editar Host, Puerto, Usuario, Contraseña y Nombre de BD.
 RESULTADO	Feedback visual inmediato del estado del servidor. Cualquier usuario puede modificar la ruta de la BD sin conocimientos de programación.

5.4.C Configuración de Hardware en Caliente (Hot-Swap IPs)

 PROBLEMA	Los routers domésticos asignan IPs dinámicas por DHCP. Si cambiaba la IP del ESP32 o la cámara, era necesario detener el programa y editar el código.
 IMPLEMENTACIÓN	En vista_configuracion.py se añadieron campos de texto para las IPs del ESP32 y la cámara, que leen/escriben directamente en ajustes_locales.json. El botón 'Aplicar' ofrece retroalimentación visual asíncrona (threading): cambia a verde durante 2 segundos.
 RESULTADO	Reconexión del hardware en caliente sin reiniciar la aplicación.

5.5 Videovigilancia Resiliente y Monitorización de Hardware

5.5.A Sistema de Tolerancia a Fallos en el Módulo CCTV

 PROBLEMA	El módulo VideoStream lanzaba excepciones abruptas cuando el ESP32-CAM se desconectaba. Sin cámara, la pantalla quedaba vacía o el programa fallaba.
 IMPLEMENTACIÓN	Se reestructuró la clase heredando de ft.Stack (capas superpuestas). Se añadió una capa Glassmorphic con icono VIDEOCAM_OFF. El motor de captura se envolvió en try/except robusto que traduce errores de red a lenguaje de usuario. La capa se muestra/oculta automáticamente.
 RESULTADO	La pestaña de cámaras ya no provoca cierres. Simula una 'pérdida de señal' típica de CCTV, informando del motivo exacto del fallo.

5.5.B Sistema de Monitorización de Salud del Hardware (Heartbeat)

 PROBLEMA	El estado del ESP32 en el Dashboard mostraba siempre 'ONLINE' de forma estática. Si se desconectaba, el programa no avisaba.
---	--

 IMPLEMENTACIÓN	Cada nueva lectura de sensores actualiza una marca de tiempo (ultima_conexion_esp32). El hilo secundario en main.py calcula la diferencia temporal; si supera el límite configurado (1-10 seg via Slider), el Dashboard muestra 'OFFLINE' en rojo.
 RESULTADO	El sistema tiene consciencia real del estado del hardware. El usuario sabe al instante si el ESP32 está transmitiendo.

5.6 Funcionalidades Avanzadas

5.6.A Control Dual de Actuadores (Automático / Manual)

 PROBLEMA	Con la integración del hardware real, los botones para alternar entre control Automático y Manual dejaron de funcionar.
 IMPLEMENTACIÓN	Se creó la función transaccional toggle_modos_actuador(uid) que invierte el estado actual en la BD. Se actualizaron callbacks en main.py y la lógica de renderizado para bloquear/desbloquear los switches correctamente.
 RESULTADO	El usuario recupera la autoridad total sobre la automatización de la comisaría.

5.6.B Auditoría y Exportación de Datos (CSV)

 PROBLEMA	El sistema histórico había perdido la capacidad de exportar datos. No existía posibilidad de hacer copias de seguridad de los chats.
 IMPLEMENTACIÓN	Se implementaron tres consultas SQL (get_all_sensores_log_csv, get_all_actuadores_log_csv, get_all_chats_csv). En main.py se programó un manejador genérico exportar_csv_genérico integrado con el FilePicker de Flet para invocar el explorador del SO.
 RESULTADO	El Comisario puede extraer en un clic toda la telemetría, el registro de actuadores y una copia de seguridad legal de los chats.

5.6.C Eficiencia Energética: Cálculo Determinista del Consumo

 PROBLEMA	El panel de consumo simulaba los vatios con un generador aleatorio (random), restando credibilidad técnica al sistema.
 IMPLEMENTACIÓN	La función determinista get_consumo_electrico() calcula el consumo real según datasheets: ESP32 activo (1.5 W), iluminación activa (15 W), ventilación activa (5.5 W), sensores activos (DHT22, LDR, MQ-2, MQ-135 a 0.8 W cada uno).
 RESULTADO	El monitor de consumo es un gemelo digital preciso. Si el usuario apaga las luces, el indicador disminuye de forma matemáticamente exacta.

5.6.D Heartbeat Elástico (Ajuste Dinámico de Responsividad)

 PROBLEMA	El timeout del Heartbeat era fijo, provocando falsos positivos 'Offline' en redes IoT lentas o con mayor intervalo de muestreo.
---	---

 IMPLEMENTACIÓN	Se añadió un Slider (1-10 segundos) en el menú de configuración que se guarda en ajustes_locales.json. El algoritmo aplica: Margen de Gracia = Intervalo Configurado + 10 segundos.
 RESULTADO	El sistema es adaptable a cualquier entorno de red, eliminando falsos positivos de desconexión.

5.7 Concurrencia, UX y Optimizaciones

5.7.A Controlador UI Optimizado mediante Threading

 PROBLEMA	El bucle de petición de datos al ESP32, si corría en el hilo de la UI, congelaba toda la ventana ('Not Responding').
 IMPLEMENTACIÓN	Arquitectura multihilo (threading nativo). El refresco de datos se derivó al hilo secundario loop_controlador_ui, que actualiza los componentes gráficos de Flet mediante callbacks asíncronos sin bloquear el hilo de renderizado.
 RESULTADO	Interfaz fluida al 100%. Las gráficas mutan en tiempo real mientras el operador interactúa con menús, chats y controles.

5.7.B Mejoras de Diseño Visual y Ergonomía

Las mejoras de UX/UI implementadas en esta fase:

- Reorganización del Histórico en grid de 3 columnas (ft.GridView, runs_count=3) para visualizar los 5 sensores sin truncamiento de fechas.
- Capa de error Glassmorphic en el módulo de cámara: placeholder translúcido con icono VIDEOCAM_OFF en lugar de pantalla negra o excepción visible.
- Autofocus en el Login: autofocus=True en el campo de usuario y reordenación del árbol de componentes para que el tabulador salte directamente a la contraseña.

5.8 Calidad de Código y Repositorio

5.8.A Manejo de Excepciones y Logging (Production-Ready)

 PROBLEMA	Las excepciones no estaban bien controladas: el terminal se llenaba de trazas confusas o los fallos eran silenciosos.
 IMPLEMENTACIÓN	Auditoría completa de manejador_datos.py y main.py. Todas las operaciones de BD y peticiones HTTP al hardware encapsuladas en try/except/finally. Mensajes de consola estandarizados con etiquetas: [ERROR BD], [ERROR LOCAL], [ERROR HARDWARE], [ERROR UI].
 RESULTADO	Código de grado de producción. Los fallos no rompen la aplicación y el logging es limpio y descriptivo.

5.8.B Optimización del Repositorio Git

 PROBLEMA	Archivos dinámicos (chats .txt, configuración .json, capturas .jpg) se subían a GitHub, ensuciando el historial y generando conflictos.
---	---

 IMPLEMENTACIÓN	Se actualizó el .gitignore para excluir: ajustes_locales.json, el directorio logs_chat/ y todos los archivos de imagen dentro de capturas_simuladas/.
 RESULTADO	Repositorio ligero y profesional. El entorno local de cada desarrollador queda completamente aislado.

5.8.C Aislamiento de Microservicios (servicios_iot/)

 PROBLEMA	El script recolector.py y los archivos .bat/.command estaban dispersos en la raíz del proyecto, mezclados con el código de la UI.
 IMPLEMENTACIÓN	Se creó el directorio servicios_iot/ para aislar los procesos de infraestructura. Se añadió un parche dinámico de rutas (sys.path.append) en el recolector y se actualizaron los scripts .bat con instrucciones cd.. para invocar correctamente el entorno virtual .venv.
 RESULTADO	Raíz del proyecto limpia y profesional. Los procesos en segundo plano se inician de forma 100% fiable.

— FASE 2 — Optimización, Seguridad y Refactorización UX/UI

5.9 Optimización de Base de Datos: Keyset Pagination

 PROBLEMA	El histórico de sensores y actuadores cargaba datos con OFFSET SQL, que obliga al motor a escanear secuencialmente todos los registros anteriores, consumiendo memoria y CPU innecesariamente.
 IMPLEMENTACIÓN	Se implementó Keyset Pagination (cursor lógico). El algoritmo memoriza la marca temporal (ultimo_timestamp) del último evento renderizado e inyecta: WHERE timestamp < ultimo_timestamp ORDER BY timestamp DESC LIMIT 50. El motor usa el índice temporal directamente.
 RESULTADO	Coste computacional constante O(1) en la recuperación de datos, independientemente del volumen histórico acumulado.

5.10 Reestructuración del Subsistema de Chat

5.10.A Refactorización Visual — Mensajería Moderna

 PROBLEMA	El chat era rudimentario: sin avatares, con texto estático '(Leído)', sin cabeceras dinámicas y sin coherencia visual con el estilo Glassmorphism del resto del software.
---	---

 IMPLEMENTACIÓN	Reprogramación completa de vista_chat.py: LEFT JOIN múltiple en dao_chat.py para extraer imagen en Base64 y cargo real (Comisario, Inspector, Policía). Sistema de Ticks adaptativos: tick gris (RECIBIDO), doble tick gris (ENTREGADO), doble tick azul (LEÍDO). Header persistente con foto de perfil circular, nombre y rol extraído de BD.
 RESULTADO	Experiencia de nivel de producción similar a WhatsApp.

5.10.B Gestión del Ciclo de Vida del Dato: Borrado de Historial

 PROBLEMA	Los mensajes se acumulaban indefinidamente. Las normativas de seguridad exigen mecanismos para purgar canales de comunicación confidenciales.
 IMPLEMENTACIÓN	Función transaccional eliminar_conversacion() en el DAO con DELETE cruzado sobre mensajes_chat. En la UI, botón de papelerita con AlertDialog modal de confirmación, recargando de forma asíncrona la lista de contactos tras la ejecución.
 RESULTADO	Control total sobre la persistencia de comunicaciones internas con salvaguarda frente a borrados accidentales.

5.11 Ciberseguridad: Blindaje BCrypt

5.11.A Reparación de Vulnerabilidad en el Proceso de Login

 PROBLEMA	La función validar_usuario tenía una condición estricta que asumía que todos los hashes debían empezar por \$2b\$ o \$2a\$. Hashes válidos bajo otro estándar (ej. \$2y\$) provocaban que el sistema pasara a comparar en texto plano, permitiendo acceso con el hash como contraseña.
 IMPLEMENTACIÓN	Se eliminó la validación estricta de prefijos y se encapsuló bcrypt.checkpw() en try/except. Solo si la cadena de BD no es un hash válido (ValueError) se ejecuta un mecanismo de contingencia (Fallback) para cuentas legacy o de prueba.
 RESULTADO	El sistema de autenticación es robusto y a prueba de fallos. Cumple con los estándares de cifrado exigibles a software de uso policial.

5.11.B Prevención de Corrupción de Datos (Doble Cifrado)

 PROBLEMA	Si un administrador editaba el perfil de un usuario y pulsaba 'Guardar' sin modificar la contraseña, el sistema volvía a encriptar el hash ya existente, destruyendo la credencial y bloqueando al agente permanentemente.
 IMPLEMENTACIÓN	Barrera algorítmica en update_usuario: antes de hacer hashing, el código verifica si el string comienza por '\$2' (firma universal de BCrypt). Si es así, lo inyecta tal cual; si no, lo cifra como nueva clave con su salt criptográfico.
 RESULTADO	El sistema previene la corrupción de credenciales por error humano, garantizando la integridad del acceso.

5.12 Limpieza de Arquitectura y Sincronización

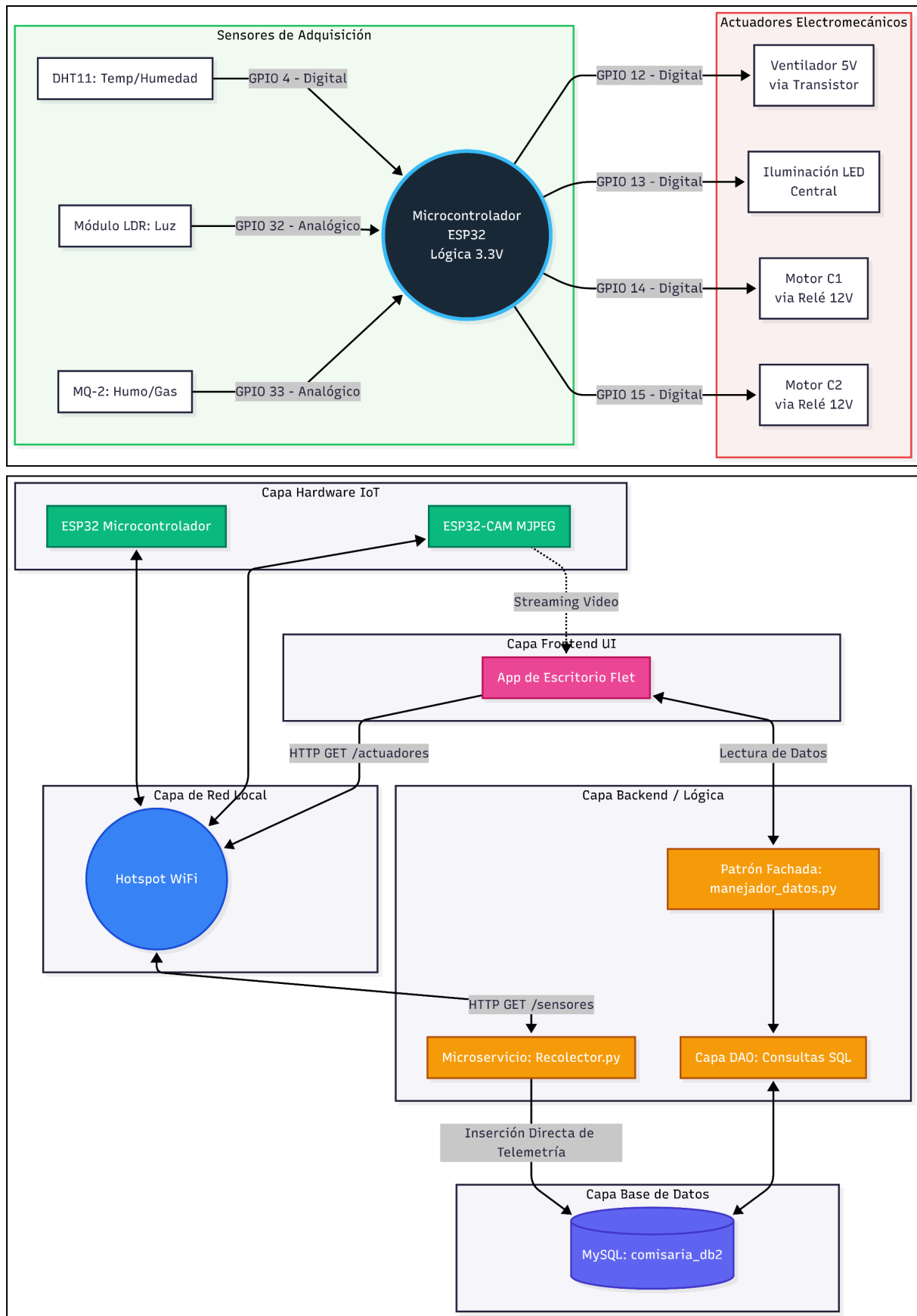
5.12.A Eliminación del Código Zombie

 PROBLEMA	Tras la refactorización DAO/Fachada, el archivo monolítico original seguía existiendo y algunos scripts dependían de él, generando un entorno frágil donde cohabitaban la arquitectura antigua y la nueva.
 IMPLEMENTACIÓN	Se purgaron definitivamente todos los archivos obsoletos. Se reescribieron las cabeceras de importación en microservicios y vistas para forzar a todos los componentes a canalizar peticiones exclusivamente a través del nuevo enrutador (modelo/manejador_datos.py).
 RESULTADO	Reducción drástica de la deuda técnica. El repositorio queda saneado y libre de colisiones de código.

5.12.B Sincronización del Gemelo Digital con el Hardware Real

 PROBLEMA	El Dashboard presentaba controles para 'Motores de Puerta' (servomotores) que no se integraron en el circuito ESP32 final, generando inconsistencia entre el frontend y el backend.
 IMPLEMENTACIÓN	Se adaptó el Dashboard desactivando y ocultando la sección de control de motores, respetando las posiciones absolutas del plano de la comisaría para no alterar la resolución visual.
 RESULTADO	El panel de control refleja con un 100% de precisión los actuadores realmente disponibles en el hardware (LEDs y Ventilador).

DIAGRAMAS DE LA APLICACIÓN Y SU ARQUITECTURA



6. VALOR DIFERENCIAL E INNOVACIÓN

6.1 Diseño Hexagonal Sincronizado con la Maqueta Real

El plano de la comisaría integrado en el Dashboard es una reproducción vectorial a escala de la maqueta física construida para el proyecto, con geometría hexagonal inspirada en la arquitectura penitenciaria panoptíctica (Jeremy Bentham, 1791). Cada zona del plano está vinculada en tiempo real a los datos de los sensores correspondientes: si un sensor de gas supera el umbral de alarma, la zona del mapa cambia de estado visualmente de forma automática. Este nivel de sincronización aplica el concepto de Digital Twin a escala educativa.

6.2 Keyset Pagination: Ingeniería de Rendimiento Aplicada

La elección de Keyset Pagination en lugar de OFFSET no es una optimización incremental: es la elección de un algoritmo asintóticamente superior. OFFSET tiene coste $O(n)$ —escala con el tamaño del histórico—; Keyset tiene coste $O(1)$ al apoyarse en el índice temporal de MySQL sin ningún escaneo previo. En un sistema que opera durante años con muestreos cada segundos, la diferencia puede ser entre una consulta instantánea y un timeout.

6.3 Interfaz Glassmorphism: Usabilidad en Entornos de Alta Tensión

El diseño Glassmorphism no es una decisión puramente estética. Los entornos de control de seguridad pública requieren interfaces que minimicen la fatiga visual durante guardias prolongadas y permitan una lectura instantánea del estado del sistema. Las capas translúcidas, los gradientes suaves y el alto contraste entre elementos activos e inactivos responden a los principios de Human Factors Engineering (HFES, ISO 9241-210:2019).



6.4 Tolerancia a Fallos: Circuit Breaker Implícito

El sistema implementa un patrón Circuit Breaker en la gestión del ESP32-CAM: ante pérdida de señal, la interfaz no colapsa ni bloquea al usuario, sino que activa una capa de placeholder informativa y reintenta la conexión en segundo plano. Comportamiento

estrictamente necesario en un entorno policial donde la aplicación debe permanecer operativa independientemente del estado del hardware periférico.

7. ESTUDIO ECONÓMICO Y MATERIALES

El presupuesto contempla tres categorías: hardware electrónico, construcción de la maqueta física y horas de ingeniería del equipo. El coste de software es cero al emplear exclusivamente herramientas de código abierto.

7.1 Coste de Hardware Electrónico (BOM — Bill of Materials)

Componente	Ud.	Precio/ud.	Total
ESP32 DevKit V1 (Espressif)	1	8,50 €	8,50 €
ESP32-CAM (OV2640)	1	7,90 €	7,90 €
Sensor DHT22 (Temp./Humedad)	1	3,20 €	3,20 €
Sensor LDR + resistencia	1	0,30 €	0,30 €
Sensor MQ-2 (Humo/Gas)	1	2,80 €	2,80 €
Sensor MQ-135 (Calidad aire)	1	3,10 €	3,10 €
Módulo relé 2 canales (12V)	1	2,50 €	2,50 €
Transistor NPN BC547 (x5)	1	0,50 €	0,50 €
Ventilador DC 5V	1	4,20 €	4,20 €
LEDs (rojo/verde/blanco) + resist.	1	0,80 €	0,80 €
Fuente alimentación 5V/12V	1	6,90 €	6,90 €
Placa protoboard + cables Dupont	1	3,50 €	3,50 €
Cableado y conectores varios	1	2,00 €	2,00 €
SUBTOTAL HARDWARE ELECTRÓNICO			46,20 €

7.2 Coste de Construcción de la Maqueta Física

Material	Cantidad	Precio	Total
Planchas de cartón pluma (80x60 cm)	4 ud.	2,50 €/ud.	10,00 €
Madera balsa (varillas 5mm)	2 paquetes	3,00 €/ud.	6,00 €
Pegamento termofusible (barras)	1 pack	3,50 €	3,50 €

Material	Cantidad	Precio	Total
Pintura acrílica (gris, blanco, negro)	3 botes	2,00 €/ud.	6,00 €
Papel de maquetación y cartulina	1 pliego	1,50 €	1,50 €
SUBTOTAL MAQUETA FÍSICA			27,00 €

7.3 Coste de Desarrollo de Software (Horas de Ingeniería)

Estimación basada en una tarifa de 15 €/hora por persona, con jornada parcial (4 horas/día, 5 días/semana) durante 3 meses (aproximadamente 12 semanas). Total por persona: 240 horas.

Integrante	Perfil	Horas	Tarifa/h	Coste
Álvaro López	Ing. Backend / Arquitectura	240 h	15,00 €/h	3.600,00 €
Daniel Nicolás Ramírez	Ing. Backend / Seguridad	240 h	15,00 €/h	3.600,00 €
Daniel Vicente	Ing. Hardware / Firmware	240 h	15,00 €/h	3.600,00 €
Fernando Fernández	Ing. Frontend / UX/UI	240 h	15,00 €/h	3.600,00 €
SUBTOTAL HORAS DE INGENIERÍA (960 h)				14.400,00 €

7.4 Resumen del Presupuesto Total

Categoría de Coste	Importe	Observaciones
Hardware electrónico (BOM)	46,20 €	Precio de mercado
Construcción maqueta física	27,00 €	Materiales
Software y licencias	0,00 €	100% Open Source
Horas de ingeniería (960 h)	14.400,00 €	4 personas × 240 h × 15 €/h
COSTE TOTAL ESTIMADO DEL PROYECTO	14.473,20 €	

Nota: El coste material real (hardware + maqueta) asciende a 73,70 €, lo que demuestra la alta eficiencia económica de la solución. El coste de ingeniería es una estimación orientativa a precio de mercado para jornada parcial de 3 meses.

8. CONCLUSIONES, AUTOCRÍTICA Y LÍNEAS FUTURAS

8.1 Conclusiones

El proyecto Qualifast Buildings ha alcanzado todos los objetivos técnicos planteados y ha superado las expectativas del equipo en varios aspectos:

- Sistema IoT de grado de producción con menos de 75 € de hardware, empleando exclusivamente herramientas de código abierto y patrones arquitectónicos consolidados.
- La arquitectura MVC+DAO+Fachada demostró su valor práctico: permitió sustituir el almacenamiento (JSON a MySQL), añadir sensores y refactorizar el chat sin modificar el resto del sistema.
- El sistema de autenticación BCrypt con blindaje contra vulnerabilidades de prefijo y doble cifrado cumple los estándares exigibles a una aplicación policial real.
- Keyset Pagination garantiza rendimiento constante tras años de operación continua con miles de registros acumulados.
- La sincronización entre el gemelo digital hexagonal y la maqueta física, junto con el chat con ticks de lectura, sitúan el proyecto en un nivel de complejidad inusual para un trabajo final de grado.

8.2 Autocrítica y Limitaciones

Red local únicamente: La arquitectura basada en Hotspot WiFi local impide el acceso remoto al sistema sin VPN o reenvío de puertos.

Sin app móvil: La interfaz es exclusivamente de escritorio (Flet). Un despliegue real requeriría una aplicación móvil o interfaz web responsiva.

Motores de puerta no implementados: Los servomotores no se integraron en el circuito final por limitaciones de tiempo y presupuesto, dejando deuda técnica en el control de puertas físicas.

Escalabilidad de la BD: MySQL en servidor local es adecuado para una única comisaría, pero un despliegue multi-sede requeriría arquitectura distribuida.

8.3 Líneas de Trabajo Futuro

Corto plazo (1-3 meses): Control de motores paso a paso (28BYJ-48 + ULN2003) para puertas físicas reales. Añadir MQTT como protocolo complementario a HTTP para telemetría en redes con alta latencia.

Medio plazo (3-12 meses): Aplicación móvil nativa (React Native o Flutter) con funcionalidades críticas del dashboard. Migración de la BD a clúster MySQL en nube (AWS RDS o Azure) con backup diario y acceso remoto vía VPN.

Largo plazo (>1 año): Módulo de IA para detección de anomalías en datos de sensores con modelos de series temporales (LSTM o Prophet). Escalado a sistema multi-sede con panel de control centralizado.

9. BIBLIOGRAFÍA Y REFERENCIAS

Referencias en formato IEEE:

[1] Espressif Systems. ESP32 Technical Reference Manual, v5.2. Espressif Systems, 2024. [En línea]. Disponible en: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[2] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA: Addison-Wesley, 2002.

- [3] Oracle Corp. 'Data Access Object', Core J2EE Patterns. [En línea]. Disponible en: <https://www.oracle.com/java/technologies/dataaccessobject.html>
- [4] N. Provos y D. Mazières, 'A Future-Adaptable Password Scheme,' in Proc. 1999 USENIX Annual Technical Conference, pp. 81-91, 1999.
- [5] M. Winand, SQL Performance Explained: Everything Developers Need to Know about SQL Performance. Markus Winand, 2012.
- [6] E. Gamma, R. Helm, R. Johnson y J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- [7] Human Factors and Ergonomics Society (HFES), ISO 9241-210:2019 — Human-Centred Design for Interactive Systems, 2019.
- [8] IoT Analytics GmbH, IoT Analytics Market Report 2024-2029. IoT Analytics, 2024. [En línea]. Disponible en: <https://iot-analytics.com/product/iot-market-report-2024/>
- [9] Python Software Foundation. Python 3.11 Documentation. [En línea]. Disponible en: <https://docs.python.org/3.11/>
- [10] Flet Inc. Flet Framework Documentation. [En línea]. Disponible en: <https://flet.dev/docs/>
- [11] MySQL AB / Oracle Corp. MySQL 8.0 Reference Manual. [En línea]. Disponible en: <https://dev.mysql.com/doc/refman/8.0/en/>
- [12] Gobierno de España, Plan de Digitalización de las Administraciones Públicas 2021-2025. Ministerio de Asuntos Económicos y Transformación Digital, 2021.